ST. PETER'S COLLEGE

COLLEGE OF COMPUTER STUDIES

WS-101 | FULL-STACK DEVELOPMENT

SEMI-FINAL PROJECT PRESENTATION & DEFENSE GUIDELINES

Assessment: 100 points | Mode: Project demonstration and oral defense

1. Overview

Students will present a working full-stack application and defend both its technical implementation and their AI-assisted development process. Coding agents are permitted, but students must explain the code, artifacts, and decisions produced during development.

2. Required Technology and Tools

- Full-stack framework: Next.js (including application routes, UI components, and server-side/data-handling logic).
- Database/backend: self-hosted Supabase running with Docker; demonstrate the application's connection to this instance.
- Coding agents: Antigravity, OpenCode, Codex, Claude, or a comparable coding agent.
- Required skills repository: Matt Pocock's Skills — https://github.com/mattpocock/skills. Students must demonstrate which skills were used and how they influenced their work.

3. What to Present

- Live application demo: show major features, navigation, CRUD operations, validation, and actual database changes.
- Architecture walkthrough: explain key Next.js directories/files, frontend-to-backend request flow, Supabase integration, and Docker services.
- Skills evidence: identify the specific skills invoked; show relevant skill-produced or skill-guided files/artifacts (for example, plans, notes, specifications, or implementation changes), where applicable. Do not claim a skill generated files if it did not.
- Development process: explain the sequence from requirements → planning → agent instructions → implementation → testing/debugging → revisions; include concrete examples of prompts and decisions.
- Code defense: open important files and explain their purpose, data flow, security controls, and how you verified the generated code.
- Questions and answers: explain why design choices were made and how bugs or incorrect AI output were addressed.

4. Required Evidence and Deliverables

- Presentation slides summarizing the problem, solution, architecture, workflow, skills used, screenshots, and lessons learned.
- Runnable project/source code with a README covering setup and basic usage.
- Evidence of skill usage and agent workflow: relevant skill names, prompts or session excerpts, generated plans/artifacts, and before/after changes when available.
- ERD or database schema overview, plus a short explanation of database tables, relationships, and access controls.
- Docker/Supabase configuration evidence; remove passwords, API secrets, tokens, and private data from screenshots and submissions.

5. Suggested Presentation Sequence

Part	Coverage
Introduction	Project purpose, target users, and key features
Live demo	Functional workflows and data changes
Technical architecture	Next.js, Supabase, Docker, database and security
Agentic workflow	Agent used, Matt Pocock skills, artifacts, prompts, debugging and revisions

Code walkthrough	Important files, request/data flow and explanation of generated code
Q&A	Technical defense and individual understanding

6. Grading Criteria (100 Points)

Criterion	Points
Full-Stack Application & Live Demo	25
Next.js, Supabase & Docker Architecture	15
Matt Pocock Skills Usage & Generated Artifacts	20
Agentic Development Workflow & Decisions	15
Code Walkthrough, Defense & Q&A	15
Database, Security & Code Quality	10
TOTAL	100

7. Rules and Reminders

- Using AI is allowed; inability to explain the resulting work will affect the defense score.
- Students must clearly distinguish what the agent generated, what a skill guided, and what they personally changed or verified.
- The application must be demonstrable. Prepare screenshots or a recording as a backup, but a working live demo is expected.
- Do not expose credentials, private business/customer data, or service keys during presentation.
- Follow the instructor's announced submission deadline, group arrangements, and presentation schedule.

Prepared by: LLOYD MILGAR T. SEMBLANTE | Instructor